UniHub

Testing Guide - step by step

For beginners. Nothing here needs programming knowledge: you will copy, paste and click.

What you will do	Time
Part 1 - One-time setup on your computer	5 min
Part 2 - Prepare Supabase (the database) in your browser	10 min
Part 3 - Start the app	5 min
Part 4 - Create your account and sample events	10 min
Part 5 - Test registration, tickets and check-in with two accounts	15 min
Part 6 - If something goes wrong	as needed

NOTE: Words in `this style` are things you type or paste exactly. Keep this PDF open next to your screen and tick off each step.

WARNING: Your C: drive is almost full. If anything fails with 'not enough space', free up space on C: first (Windows Settings > System > Storage > Temporary files). The project itself lives on D: and is fine.

Part 1 - One-time setup

What is PowerShell? A window where you type commands. You will use it to start the app. It is already on your computer.

1. Press the **Windows key**, type `PowerShell` and press **Enter**. A blue or black window opens.

2. Tell Flutter to keep its downloads on the D: drive (so C: does not fill up). Paste this line and press Enter. You only ever do this once:

```
setx PUB_CACHE D:\pub-cache
```

3. **Close the PowerShell window and open a new one** (same as step 1). The setting only applies to new windows.

4. Check that Flutter works. Paste this and press Enter:

```
flutter --version
```

You should see a line starting with *Flutter 3.* followed by numbers. If you see *'flutter' is not recognized*, see Part 6.

Part 2 - Prepare Supabase

Supabase is the online database that stores accounts and events. You work with it in your web browser. Your project address is:

```
https://supabase.com/dashboard/project/xwdaygcezdbneoubimgv
```

Log in to Supabase if it asks you to.

2A. Add HAW Hamburg (fixes 'no university account found')

The database only knew TUM, LMU and Uni Hamburg. HAW Hamburg (@haw-hamburg.de) is a different university, so it was missing. This step adds it and also fixes the account you already created.

1. In PowerShell, open the SQL file in Notepad:

```
notepad D:\UniHub_v0.2\supabase\migrations\20261020010000_add_haw_hamburg.sql
```

2. In Notepad press **Ctrl+A** (select all), then **Ctrl+C** (copy). You can close Notepad.

3. In your browser open the **SQL Editor**:

```
https://supabase.com/dashboard/project/xwdaygcezdbneoubimgv/sql/new
```

4. Click into the big empty box, press **Ctrl+A** then **Ctrl+V** to paste, and click the green **Run** button (or press Ctrl+Enter).

5. At the bottom you should see **Success** (it may show a small table with one row). If you see red text, copy it and send it to me.

2B. Make the sign-up email contain a code

The app asks you to type a 6-digit code from an email. Supabase only sends a link by default, so we change the email text once.

1. Open the email templates page:

```
https://supabase.com/dashboard/project/xwdaygcezdbneoubimgv/auth/templates
```

2. Click **Confirm sign up** in the list.
3. You will see a **Subject** box and a **Message body** editor. If you see two tabs, choose the one called **Source** (not Preview). Click in the body, press Ctrl+A to select the old text, and paste this:

```
<h2>Your UniHub code</h2>
<p>Enter this code in the app: <b>{{ .Token }}</b></p>
```

4. Click **Save** (bottom right of the template).
5. Do exactly the same for the template called **Reset password**.

TIP: {{ .Token }} is a placeholder. Supabase swaps it for the real 6-digit code when it sends the email. Type it exactly, including the double curly brackets.

NOTE: Supabase's built-in email sender allows only a few emails per hour. If a code does not arrive, wait a few minutes and check your spam folder before trying again.

2C. Check that email confirmation is on

1. Open `https://supabase.com/dashboard/project/xwdaygcezdbneoubimgv/auth/providers` and click **Email**.
2. Make sure **Confirm email** is switched **ON**. This is what proves someone really owns their @haw-hamburg.de address, so please leave it on.

Part 3 - Start the app

We run the app as a website on your own computer. We use a fixed address so you can open it in two browser windows (needed in Part 5).

1. Open a **new** PowerShell window.
2. Go to the project folder:

```
cd D:\UniHub_v0.2
```

3. Start the app:

```
flutter run -d web-server --web-port 8080
```

4. Wait 1 to 3 minutes the first time. Lines scroll by. When you see *'lib\main.dart is being served at http://localhost:8080'* it is ready.
5. Open your normal browser (Chrome) and go to:

```
http://localhost:8080
```

You should see the **Welcome to UNIHUB** sign-in screen.

> **WARNING:** Do not close the PowerShell window while you are testing. Closing it stops the app. To stop the app on purpose, click the PowerShell window and press **q**.

> **TIP:** After a change, click the PowerShell window and press a capital **R** (Shift+r) to restart the app quickly, then refresh the browser tab.

Part 4 - Your account and sample events

4A. Sign up (account A, your university account)

1. On the sign-in screen click **New here? Create an account**.
2. Enter your name, your university email (who649@haw-hamburg.de) and a password of at least 8 characters.
3. While you type the email you should see: *University account detected: Hamburg University of Applied Sciences*. If it says 'No university email detected', the app is out of date - stop the app (q) and start it again with the commands from Part 3.
4. Click **Create account**. Check your email inbox (and spam) for the code, type it in and click **Verify**.

> **NOTE:** You already created this account earlier. If the email says the user exists, use **Sign in** instead. After running the SQL in 2A your account is linked to HAW Hamburg automatically; sign out and in once to see it.

4B. Make account A an organizer and add sample events

New accounts are students. Only approved organizers can publish events, so we approve you directly in the database (only you, as the project owner, can do this).

1. Open the file in Notepad:

```
notepad D:\UniHub_v0.2\supabase\dev_seed_events.sql
```

2. Ctrl+A, Ctrl+C, then paste into the SQL Editor (same page as in 2A) and click **Run**.

3. You should see **Success. No rows returned**. If it says 'No profile found', you have not signed up yet (4A).

4. Back in the app, **sign out and sign in again** (account menu in the top right > Sign out). You are now in the **Planner** view with Dashboard, Create Event and Scanner.

> **TIP:** An organizer sees only the planner screens - not the student feed. That is why Part 5 uses a second account.

Part 5 - Test registration, tickets and check-in

You need two people: the **organizer** (account A) and a **student** (account B). You can be both by using two browser windows that do not share logins.

5A. Create account B (the student)

1. Open a **private / Incognito window** in Chrome (Ctrl+Shift+N) and go to `http://localhost:8080`.

2. Create a second account with a different email you can receive. A trick for Gmail: add `+student` before the @, for example `whp2718+student@gmail.com`. It arrives in the same inbox.

3. Enter the code from the email. You are in the student feed.

> **NOTE:** This account has no university, so it only sees **public** events. That is expected: it is the 'Public Events' tab. The sample events 'Isar Summer Music Festival' and 'Student Group Run' are public.

5B. Student registers and gets a ticket (window B)

1. Click the **Public Events** tab. Pull down to refresh if the list is empty.

2. On **Student Group Run** click **RSVP**. A green message says you are registered.

3. Open the **My Tickets** tab. You see your ticket with a QR code.

4. At the bottom of the ticket, small grey text says `debug code: ...`. Select all of that long code with the mouse and copy it (Ctrl+C). This stands in for scanning, because you only have one computer.

5C. Organizer checks the ticket in (window A)

1. Go to the normal browser window (account A) and open the **Scanner** tab.

2. At the top, choose **Student Group Run** from the drop-down.

3. Click the **keyboard icon** in the top bar (tooltip 'Enter code manually'), paste the code, click **Check in**.

4. A green sheet says **Access Granted** and shows the student's name.

5. Do step 3 again with the same code. It must now say the guest **was already checked in** (red). That proves one ticket cannot be used twice.

6. Back in window B, open My Tickets and click the refresh icon. The ticket now says **CHECKED IN**.

5D. Optional: test the waitlist

1. In the SQL Editor run this to limit the event to one seat:

```
update public.events set capacity = 1 where title = 'Isar Summer Music Festival';
```

2. Window B registers for 'Isar Summer Music Festival' and gets a ticket. Create a third account (for example +student2) and register: it says the event is full and you are on the **waitlist**.

3. Window B cancels (My Tickets > Cancel registration). The waitlisted account gets the ticket automatically after a refresh.

What is still fake (coming next)

Area	Status
Reviews	Stored on the device only; not yet in the database

Area	Status
Planner dashboard numbers and bars	Sample numbers, not real counts yet
Event images on events you create	No upload yet; a stock photo is used
Paid tickets	Not supported; free events only

Part 6 - If something goes wrong

Find your message in the left column. If it is not listed, send me the **last 30 lines** from the PowerShell window or a screenshot of the browser, plus what you clicked just before.

What you see	What to do
'flutter' is not recognized	Flutter is not on your PATH in this window. Close PowerShell, open a new one and try again. Still failing? Run `C:\src\flutter\bin\flutter --version`. If that works, tell me and I will fix your PATH.
not enough space on the disk	C: is full. Free space on C: (Settings > System > Storage > Temporary files), then run the command again.
PowerShell seems stuck for many minutes	The first start can take 3 minutes. If nothing changes for 10 minutes, click the window, press Ctrl+C, and run the start command again.
Page at localhost:8080 is blank or white	Wait 20 seconds, then refresh (F5). Still blank: press Ctrl+Shift+R. Then look at the PowerShell window for red text.
Port 8080 already in use	An old copy is still running. Close the old PowerShell window, or use another number: `flutter run -d web-server --web-port 8090` and open localhost:8090.
No email / code never arrives	Wait 2 minutes, check spam. Supabase limits emails per hour. Check that you saved the template in 2B. Use 'Resend code' once, then wait.
'That code is wrong or has expired'	Use the newest email only. Codes expire after a while. Click 'Resend code' and use the newest one.
Still says 'No university email detected'	The app was started before the update. Press q in PowerShell, run `cd D:\UniHub_v0.2`, then the start command again. Hard refresh the browser (Ctrl+Shift+R).
Signed in but the feed is empty	Pull down to refresh. For account B only public events show. For account A, run the sample events SQL (4B).
'Could not load events'	No internet or Supabase is unreachable. Check your connection, click Try again. If it persists, send me the PowerShell output.
'Your account is not approved to publish events yet'	That account is a student. Run the organizer SQL (4B) and then sign out and in again.
'Registration failed' or 'event is not available'	The event may have ended, or it belongs to another university than the account. Try a public event.
Sample-events SQL says 'No profile found'	Sign up in the app first (4A), then run it again.
Scanner shows 'not allowed to scan'	Only the account that created the event can check people in. Use account A.

Words explained

Word	Meaning
Supabase	The online service that holds your database, accounts and security rules.
SQL Editor	A page in Supabase where you paste instructions for the database (the .sql files).
Tenant	A university. The app shows each student their own university's events.
Organizer	An approved account that may publish events and check guests in.

Word	Meaning
Waitlist	A queue for full events. People move up automatically when someone cancels.
Hot restart (R)	Reloads the app with your latest code without starting everything from zero.